

Saving Game Data in Unity

유니티에서 게임 데이터 저장하기

CopyRight ® 원광대학교 디지털콘텐츠공학과

- You will learn how to store and retrieve data for your Unity game







CopyRight ® 원광대학교 디지털콘텐츠공학과

- You will learn how to store and retrieve data for your Unity game







CopyRight ® 원광대학교 디지털콘텐츠공학과

- Data Save and Load
  - PlayerPrefs : 유니티에서 제공하는 클래스
  - JSON (JavaScript Object Notation)
  - PlayFab
  - Database

CopyRight ® 원광대학교 디지털콘텐츠공학과

(1) PlayerPrefs

- PlayerPrefs 클래스 : 유니티에서 제공하는 데이터 관리 클래스
- int, float, string, bool 타입의 변수를 Save/Load하는 기능을 제공



CopyRight ® 원광대학교 디지털콘텐츠공학과

- DataManagerWithPlayerPrefs() 스크립트

```
...
using UnityEngine.UI;

public class DataManagerWithPlayerPrefs : MonoBehaviour {
    public InputField inputName;
    public InputField inputCount;
    public InputField inputScore;

    // Start is called before the first frame update
    void Start() {}

    // Update is called once per frame
    void Update() {}

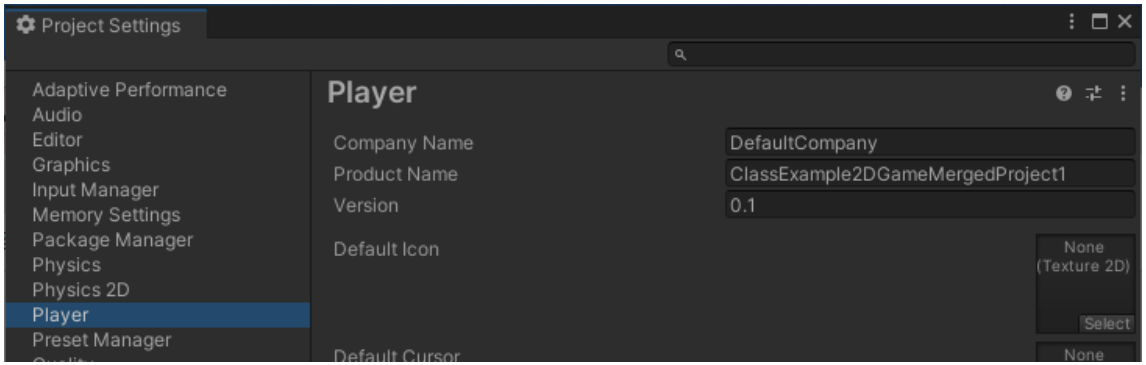
    // Write data into Registry
    public void Save() {
        PlayerPrefs.SetString("Name", inputName.text);
        PlayerPrefs.SetInt("Count", int.Parse(inputCount.text));
        PlayerPrefs.SetInt("Score", int.Parse(inputScore.text));
    }

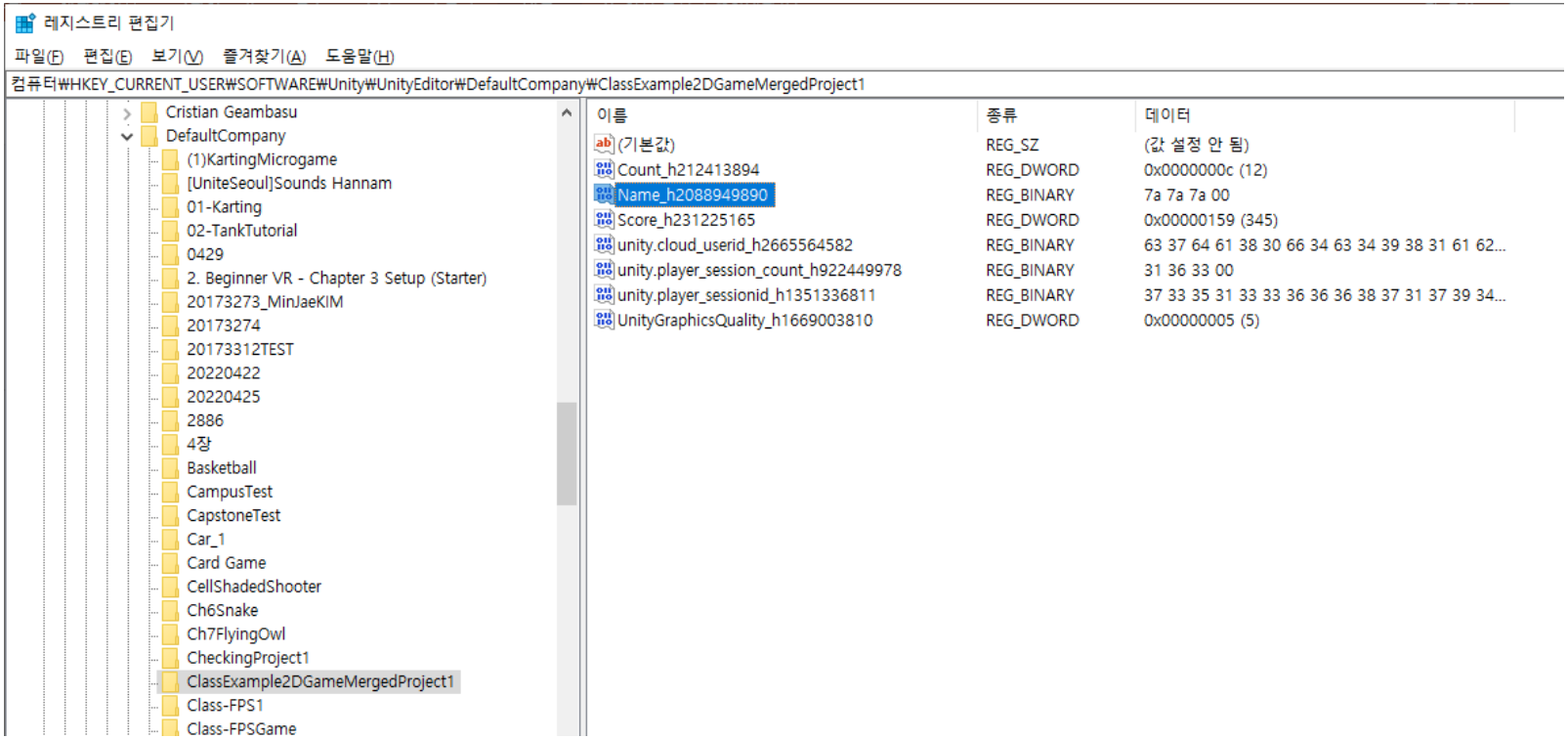
    // Read data from Registry
    public void Load() {
        if(PlayerPrefs.HasKey("Name")) {
            string getName = PlayerPrefs.GetString("Name");
            int getCount = PlayerPrefs.GetInt("Count");
            int getScore = PlayerPrefs.GetInt("Score");

            print(getName + " " + getCount + " " + getScore);
        }
    }
}
```

CopyRight ® 원광대학교 디지털콘텐츠공학과

[레지스트리 편집기]
HKEY_CURRENT_USER → SOFTWARE → Unity → UnityEditor → [Company Name] → [Product Name]





CopyRight ® 원광대학교 디지털콘텐츠공학과

- Data Save and Load
  - PlayerPrefs
    - 유니티에서 제공하는 클래스를 사용
    - 가독성이 떨어짐
    - 저장하는 데이터 타입에 제한적임
    - 보안상 문제가 있어, 해킹 당할 수도 있음
  - JSON
  - PlayFab
  - Database

CopyRight ® 원광대학교 디지털콘텐츠공학과

- Data Save and Load
  - PlayerPrefsExtra
    - Unity PlayerPrefsExtra gives you the ability to save more complex data types such as : Vectors, Bools, Colors, Lists .. , and it uses the Unity's PlayerPrefs behind the scene



CopyRight ® 원광대학교 디지털콘텐츠공학과

- Data Save and Load
  - PlayerPrefs
  - JSON (JavaScript Object Natation)
    - 경량의 데이터 교환 형식 (a lightweight format for storing and transporting data)

    (1) 유니티엔진의 JsonUtility 클래스를 이용하여 오브젝트(클래스, 데이터)를 JSON 형식의 문자열로 변환

    (2) 외부 라이브러리를 사용하여 오브젝트를 JSON 형식의 문자열로 변환

    - 변환된 문자열을 Store/Load
  - PlayFab
  - Database

CopyRight ® 원광대학교 디지털콘텐츠공학과

- JSON (JavaScript Object Natation)
  - 경량의 데이터 교환 형식 (a lightweight format for storing and transporting data)
  - is often used when data is sent from a server to a web page
  - 자바스크립트 객체와 같이 key/value가 존재하며, key값이나 문자열은 항상 쌍따옴표(“”)를 이용하여 표기
  - 데이터의 Key에 대응하는 Value 집합으로 작성 (읽거나 수정하기가 편리)
  - 객체, 배열 등 표기할 수 있으며, 원하는 만큼 중첩시켜서 사용할 수 있음

CopyRight ® 원광대학교 디지털콘텐츠공학과

- Example .json File

score.json

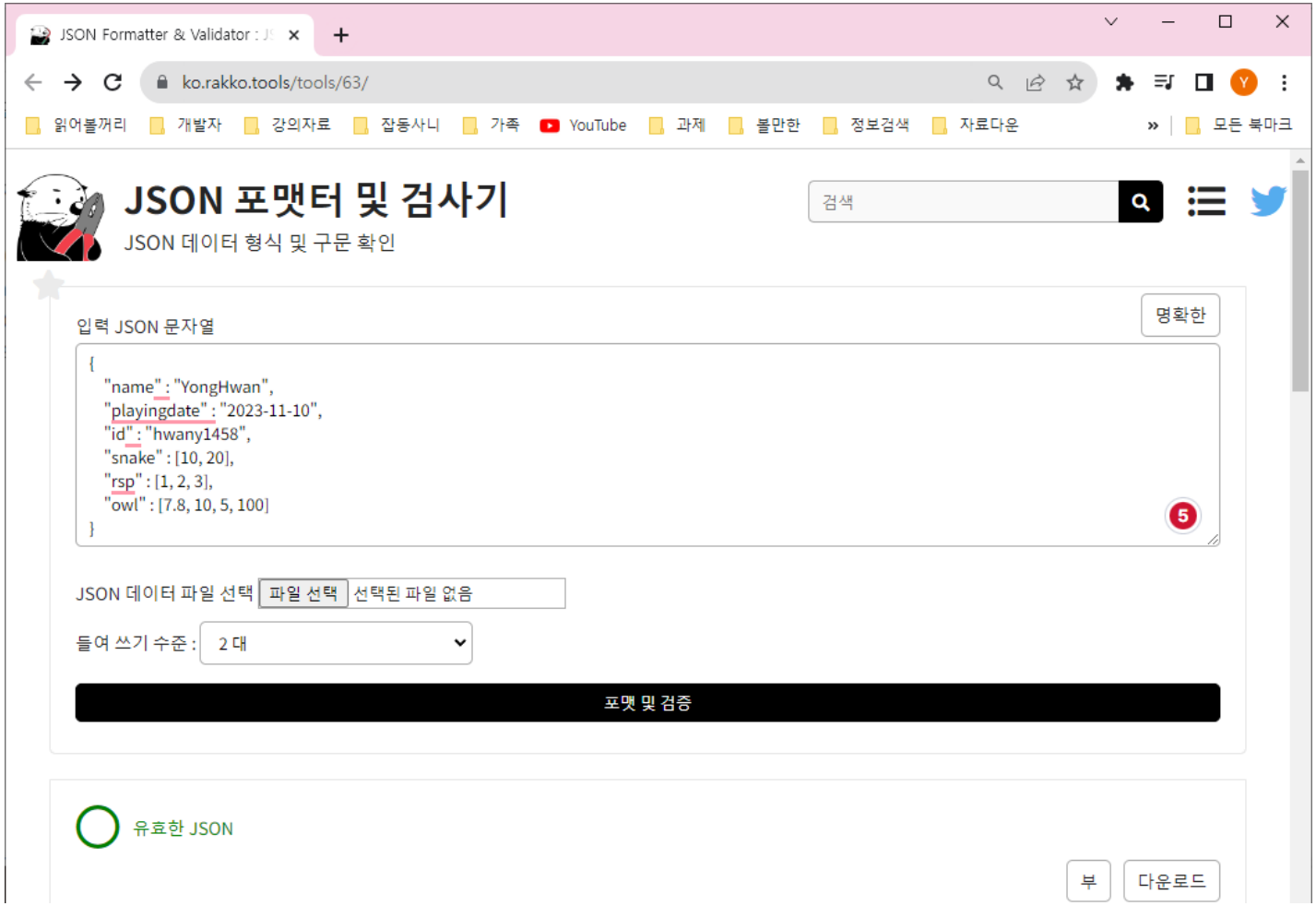
```
{
    "name" : "YongHwan",
    "playingdate" : "2023-11-10",
    "id" : "hwany1458",
    "snake" : [10, 20],
    "rsp" : [1, 2, 3],
    "owl" : [7.8, 10, 5, 100]
}
```

test.json

```
{
    "name" : "YongHwan Lee",
    "affiliation" : "Wonkwang University",
    "gender" : "Male",
    "age" : 50,
    "lectures" : [
        "C Language",
        "Advanced Programming Language",
        "HTML5 Programming",
        "Understanding to Game Engine",
        "Game Programming",
        "Mixed Reality"
    ]
}
```

CopyRight ® 원광대학교 디지털콘텐츠공학과

- Google [json 파일 포맷 검사]로 검색



CopyRight ® 원광대학교 디지털콘텐츠공학과

- Save Data to JSON, and Load it from JSON

  (1) Unity Engine에서 제공하는 JsonUtility 를 사용하는 방법

  (2) 외부 라이브러리를 사용하는 방법

    - LitJSON
    - Newtonsoft.json

CopyRight ® 원광대학교 디지털콘텐츠공학과

- Save Data to JSON, and Load it from JSON

(1) Unity Engine에서 제공하는 JsonUtility 를 사용하는 방법

– 저장하려는 데이터 포맷 생성

- Player 오브젝트 생성
- Player에 대한 여러 데이터를 PlayerData 라는 객체로 저장 → 파일로 저장
- PlayController 스크립트를 생성하고 Player 오브젝트에 연결

  (Player 오브젝트에 PlayerController 컴포넌트 생성)

– 유니티 오브젝트 정보를 JSON 파일로 저장



CopyRight ® 원광대학교 디지털콘텐츠공학과

- DataManagerWithJsonUtility 스크립트

```
...

public class DataManagerWithJsonUtility : MonoBehaviour {
    // 유니티와 JSON이 서로 주고받을 실질적인 데이터
    public PlayerData playerData;

    // Start is called before the first frame update
    void Start() {}

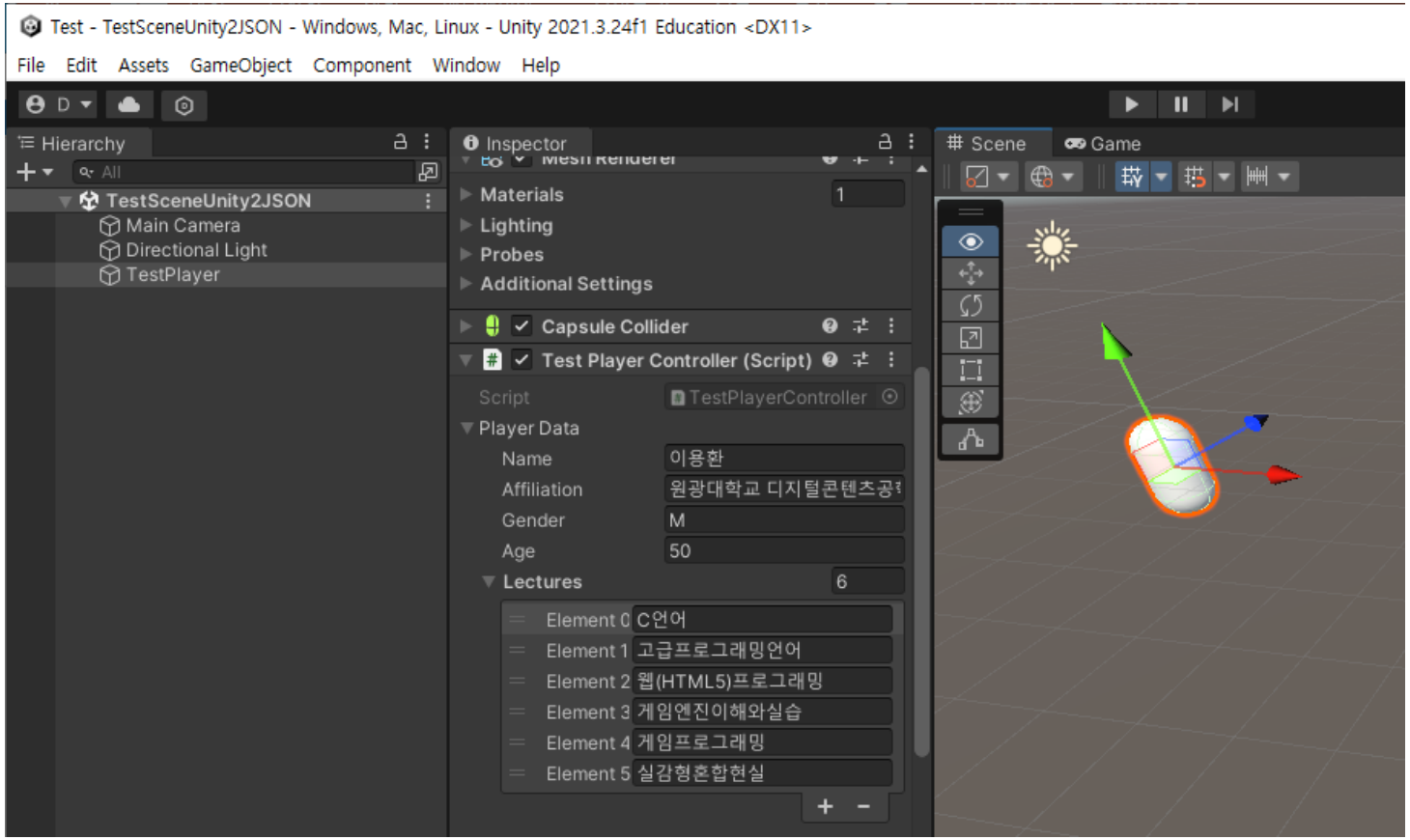
    // Update is called once per frame
    void Update() {}
}

// Player 정보만 저장할 단순 컨테이너 클래스를 생성
// Serialize 되어 있지 않기 때문에 인스팩터에 나타나지 않음
[System.Serializable]
public class PlayerData {
    // Player에 필요한 데이터를 명시
    public string name;
    public string affiliation;
    public string gender;
    public int age;
    public string[] lectures;
}
```

- PlayerController 와 PlayerData 를 분리

(1)
코드 간결성 : 데이터를 가지고 있는 컨테이너(Data Container)와
데이터를 가지고 실제로 무언가를 실행하는 컨트롤러(Controller)를 분리되어 있어야 코드가 간결해짐
(2)
PlayerController는 MonoBehaviour에서 상속받기 때문에,
PlayerController 스크립트를 Serialize화 해서 JSON 파일로 저장하게 되면,
Player 정보 뿐만 아니라 불필요한 MonoBehaviour 정보까지 함께 저장될 수 있음

CopyRight ® 원광대학교 디지털콘텐츠공학과



CopyRight ® 원광대학교 디지털콘텐츠공학과

- ToJson() 스크립트

```
...
using System.IO;  // 파일 처리 기능을 추가

public class DataManagerWithJsonUtility : MonoBehaviour {
    // 필요한 실질적인 데이터 (객체로부터 제공받음)
    public PlayerData playerData;

    ...

    //--------------
    // JsonUtility를 이용하여 오브젝트 데이터를 JSON 데이터로 저장
    // (PlayerController에서 처리하는 게 아니라, DataManager에서 처리하는 게 더 좋음)
    // 여기서는 ContextMenu를 사용
    // 컴포넌트의 뒤에 함수를 실행하는 메뉴버튼을 추가
    [ContextMenu("To Json Data")]
    public void WriteToJson() {
        // 유니티에 내장된 JsonUtility 사용
        // Json 문자열로 변형하고 싶은 오브젝트를 넘겨줌 (Json 형태로 포맷팅된 문자열이 나옴)
        // 결과를 문자열로 받음
        string jsonData = JsonUtility.ToJson(playerData, true);

        // Json 데이터를 파일로 저장
        // 현재 실행중인 유니티 프로젝트 경로에 저장 : Application.dataPath
        string writeFileName = Path.Combine(Application.dataPath, "playerData.json");
        File.WriteAllText(writeFileName, jsonData);
    }

}
```

CopyRight ® 원광대학교 디지털콘텐츠공학과

- FromJson() 스크립트

```
...
using System.IO;  // 파일 처리 기능을 추가

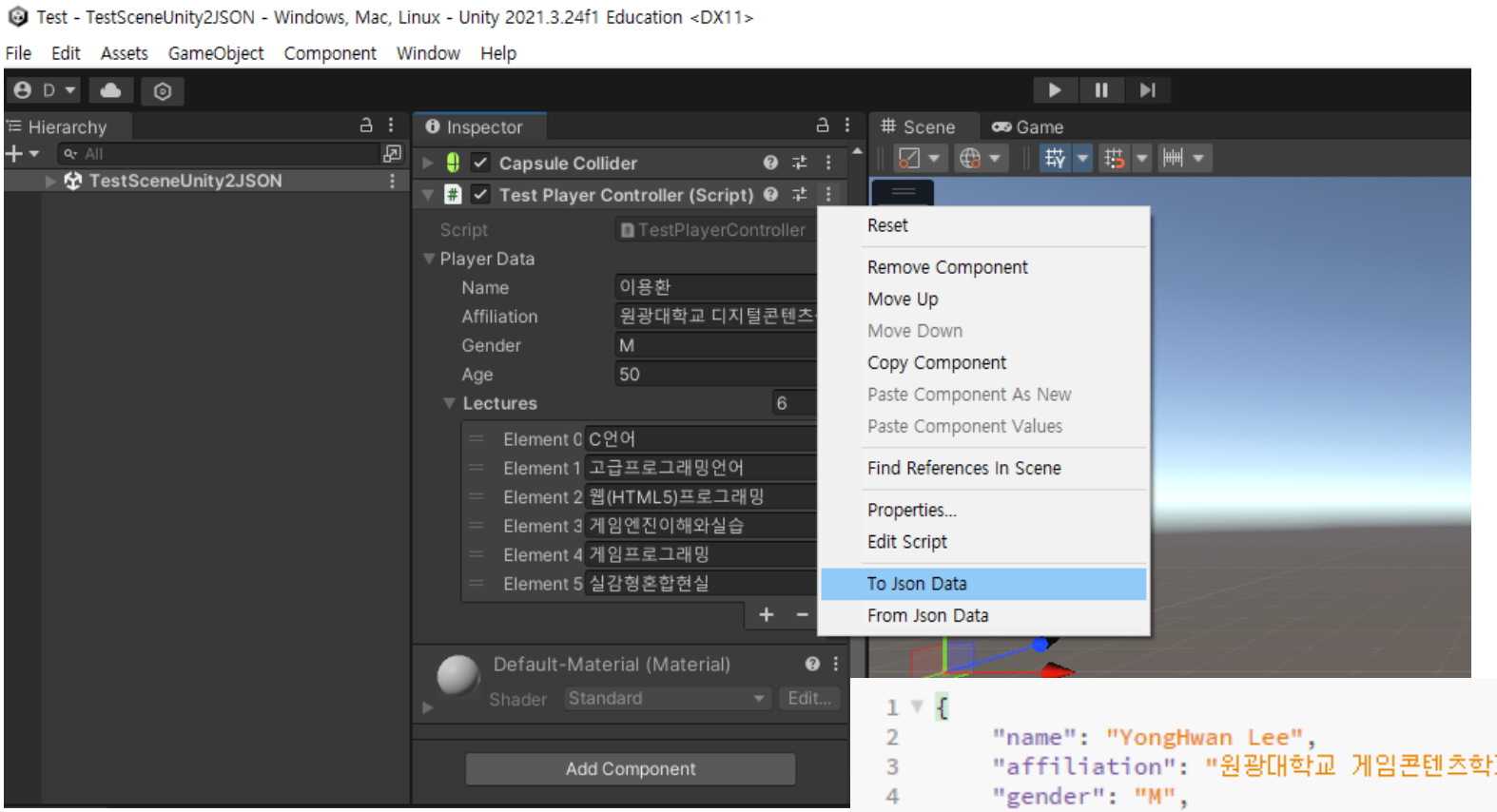
public class DataManagerWithJsonUtility : MonoBehaviour {
    // 필요한 실질적인 데이터 (객체로부터 제공받음)
    public PlayerData playerData;

    ...

    //--------------
    [ContextMenu("From Json Data")]
    public void LoadPlayerDataFromJson()
    {
        // 현재 실행중인 유니티 프로젝트 경로에 저장 : Application.dataPath
        string readFileName = Path.Combine(Application.dataPath, "playerData.json");
        // JSON 데이터를 가져와서, JSON 데이터로부터 오브젝트를 생성해서 데이터를 써줌
        // 해당경로로부터 문자열 데이터를 읽어서 문자열로 반환
        string jsonData = File.ReadAllText(readFileName);

        // JsonUtility의 FromJson을 통해 JSON으로부터 오브젝트로 Deserialize
        // 오브젝트로 복구시킬 대상을 제네릭으로 명시해야 함
        // (JSON 데이터를 PlayerData 오브젝트로 복구시킴)
        // 출력으로 나온 PlayerData 타입의 오브젝트를 PlayerController가 가졌던 PlayerData로 덮어쓰기
        playerData = JsonUtility.FromJson<PlayerData>(jsonData);
    }
}
```

CopyRight ® 원광대학교 디지털콘텐츠공학과




```
{
    "name": "YongHwan Lee",
    "affiliation": "원광대학교 게임콘텐츠학과",
    "gender": "M",
    "age": 40,
    "lectures": [
        "C언어",
        "고급프로그래밍언어",
        "웹(HTML5)프로그래밍",
        "게임엔진이해와실습",
        "게임프로그래밍",
        "실감형혼합현실",
        "캡스톤디자인1",
        "대학생활과자기혁신"
    ]
}
```


CopyRight ® 원광대학교 디지털콘텐츠공학과

- Data Save and Load
  - JSON
    - JSON은 개방형 표준 포맷으로, 텍스트를 사용하기 때문에 **사용자가 이해**하기 쉽다
    - 변환이 쉽고 사용하기 간편
    - 네트워크 게임을 개발할 때, 게임에 필요한 데이터를 주고 받거나 게임 진행 상황을 저장하거나 게임 설정을 저장하는 방식으로 많이 사용
    - 유니티에서 기본으로 제공하는 JsonUtility는 컴팩트한 최소 기능만을 제공하기 때문에 JSON 라이브러리의 모든 기능을 사용하려면 다른 JSON 라이브러리 사용을 권장
    - XML을 사용하는 것과 유사하지만, XML은 가독성이 떨어지고 데이터를 넣고 꺼낼 때 파싱(Parsing)하는 과정이 까다로움
    - 반면에, JSON은 (XML에 비해) 가독성이 좋고 직렬화(Serialize)/비직렬화 (Deserialize) 함수를 통해 데이터에서 JSON 데이터로, JSON 데이터에서 데이터로 편하게 변환할 수 있는 장점이 있음

CopyRight ® 원광대학교 디지털콘텐츠공학과

- Save Data to JSON, and Load it from JSON

  (1) Unity Engine에서 제공하는 JsonUtility 를 사용하는 방법

  (2) 외부 라이브러리를 사용하는 방법

  - LitJSON
  - Newtonsoft.json

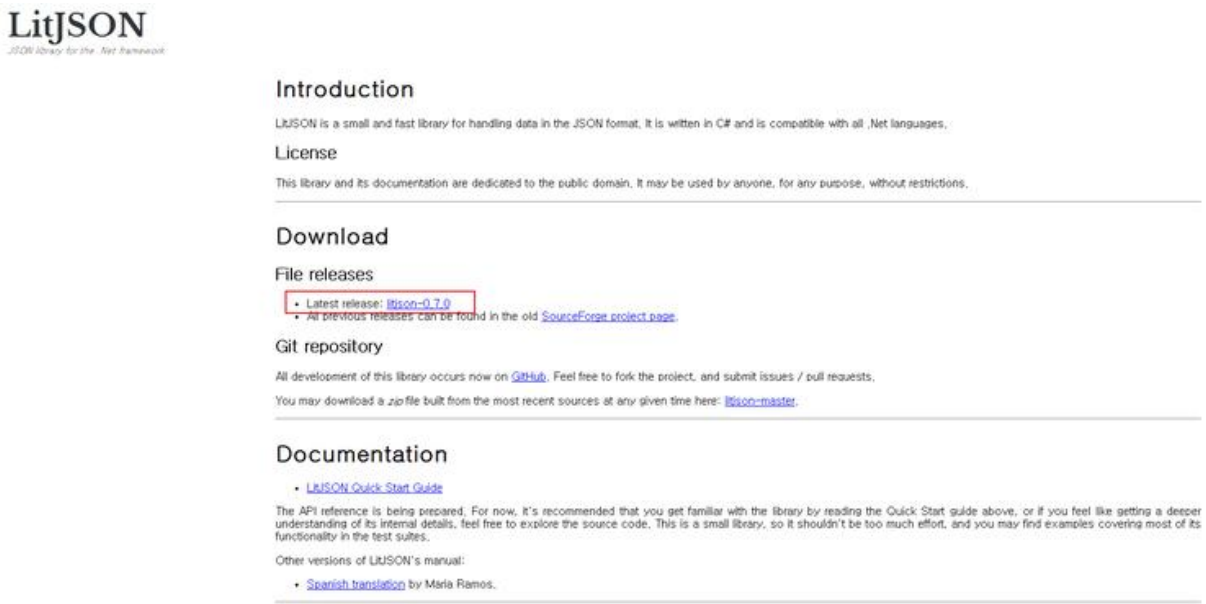
CopyRight ® 원광대학교 디지털콘텐츠공학과

- Save Data to JSON, and Load it from JSON

(1) Unity Engine에서 제공하는 JsonUtility 를 사용하는 방법

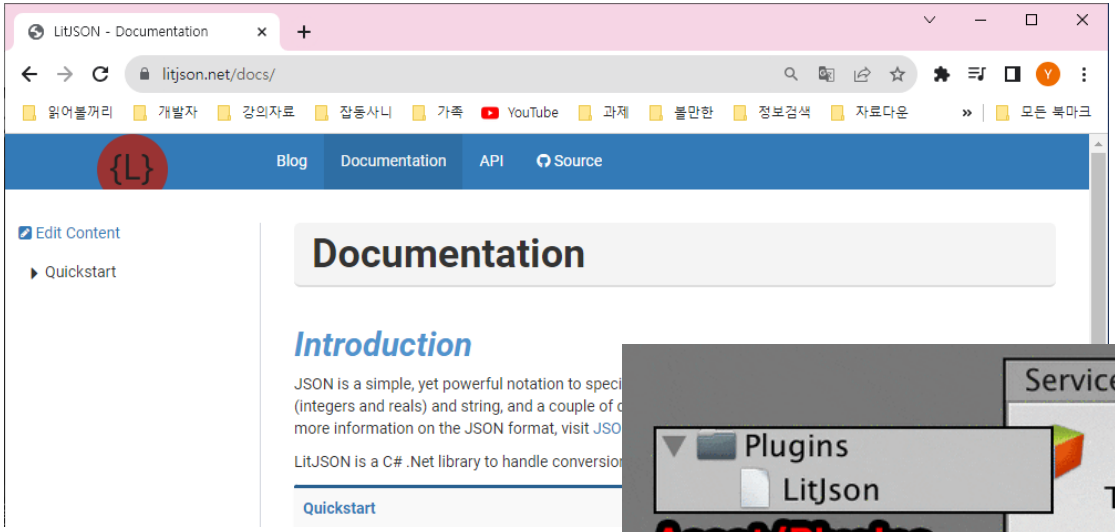
(2) 외부 라이브러리를 사용하는 방법

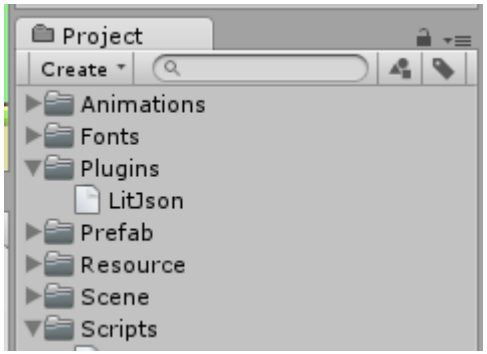
- LitJSON
  - 인터넷에서 검색하여 DLL파일을 다운
- Newtonsoft.json

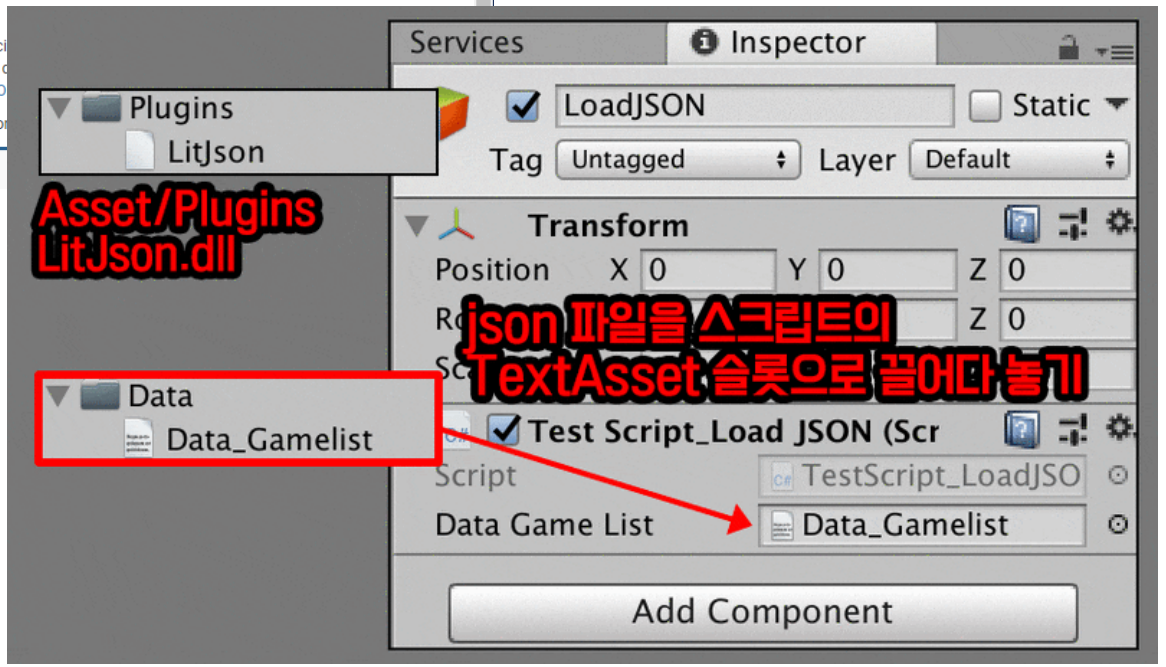


CopyRight ® 원광대학교 디지털콘텐츠공학과

– 라이브러리 파일 Import



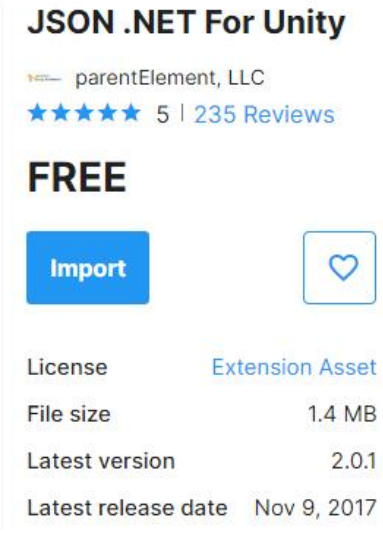




CopyRight ® 원광대학교 디지털콘텐츠공학과

- Save Data to JSON, and Load it from JSON
  - (1) Unity Engine에서 제공하는 JsonUtility 를 사용하는 방법
  - (2) 외부 라이브러리를 사용하는 방법
    - LitJSON
    - newtonsoft.json







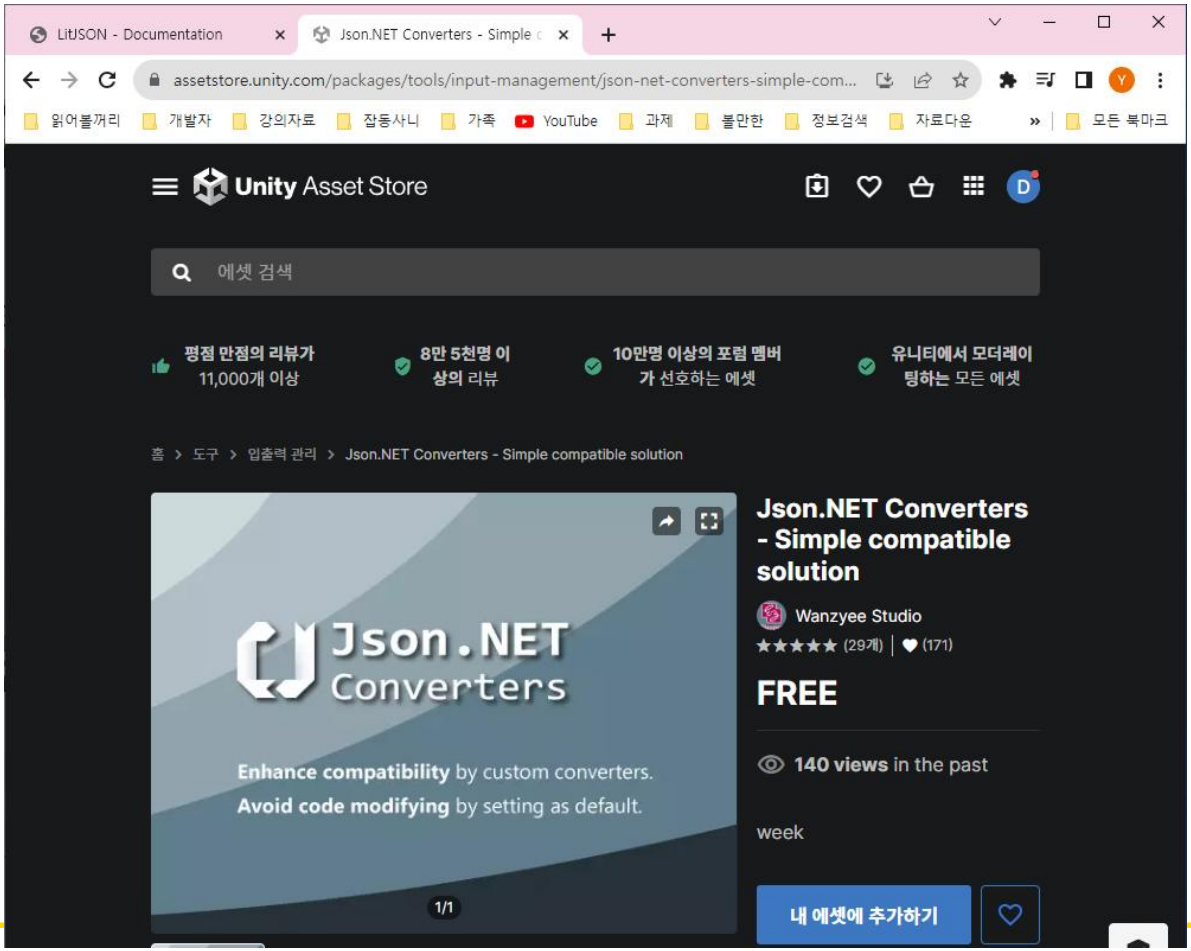
CopyRight ® 원광대학교 디지털콘텐츠공학과

- Save / Load between Unity – JSON file
  - Unity Package Manager 를 이용하여 Newtonsoft.Json 설치
  - Unity – Json 직렬화(Serialize)
  - Json – Unity 역직렬화(Deserialize)

CopyRight ® 원광대학교 디지털콘텐츠공학과

– Save / Load between Unity – JSON file

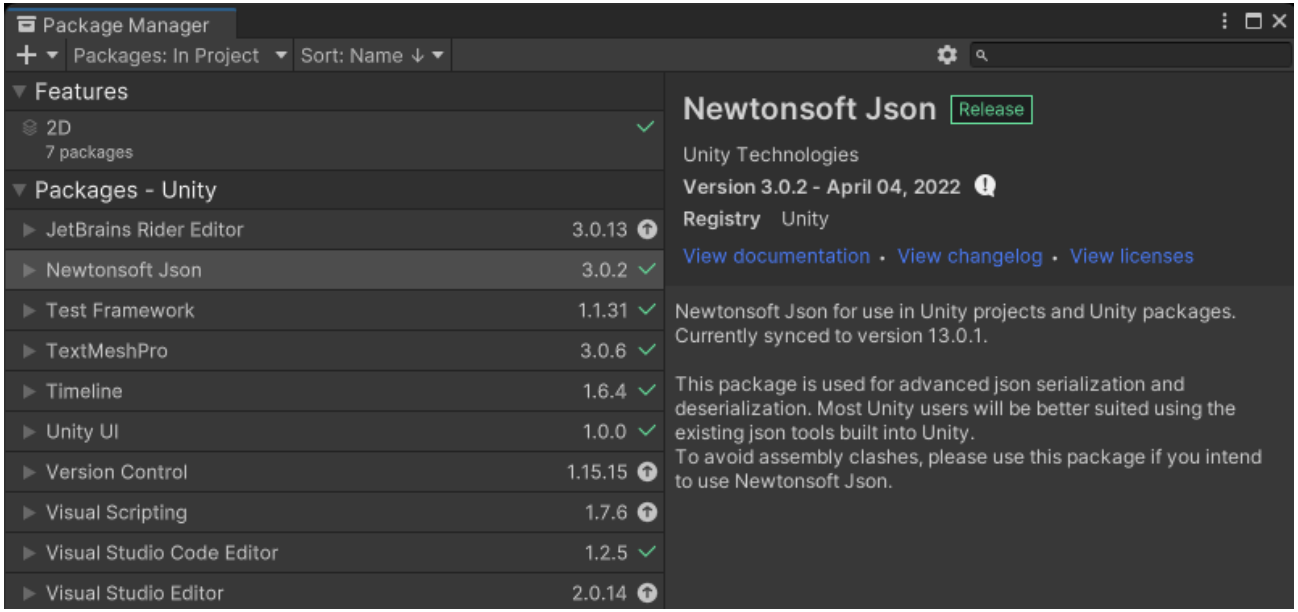
- Unity Package Manager 를 이용하여 Newtonsoft.Json 설치



CopyRight ® 원광대학교 디지털콘텐츠공학과

– Unity Package Manager 에서 JSON 라이브러리를 설치

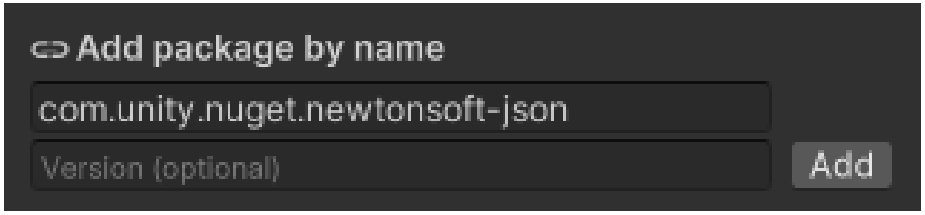
  • [Window – Package Manager] 를 클릭



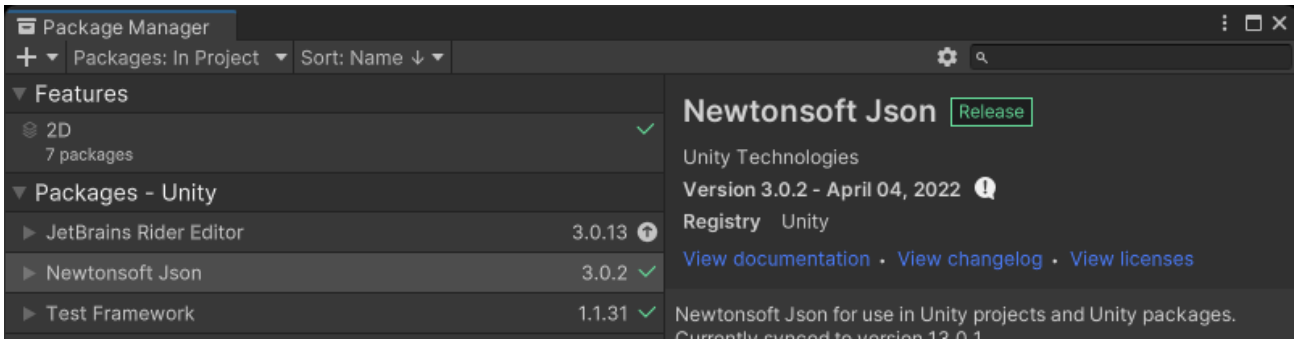
CopyRight ® 원광대학교 디지털콘텐츠공학과

– Unity Package Manager 에서 JSON 라이브러리를 설치

- Package Manager 에서 좌측 상단 “+” 버튼을 클릭하고 “Add package by name”을 선택하여 신규 패키지를 설치



`com.unity.nuget.netwonsoft-json`



CopyRight ® 원광대학교 디지털콘텐츠공학과

- Unity JSON Serialize (직렬화)
  - 새로운 클래스를 만들고 코드를 작성
  - Object to Json : SerializeObject()
  - Json to Object : DeserializeObject<클래스형식>()

- Serialize 처리 부분

```
...
using Newtonsoft.Json;

public class TestDataManager : MonoBehaviour {
    People p1 = new People("이용환", 50);

    // Start is called before the first frame update
    void Start() {
        string jData = JsonConvert.SerializeObject(p1);
        string savingJsonFileName = Path.Combine(Application.dataPath, “Object2Json.json”);
        File.WriteAllText(savingJsonFileName, jData);
        //print(jData);
    }

    // Update is called once per frame
    void Update() {}
}

public class People {
    public string name;
    public int age;

    public People(string name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

CopyRight ® 원광대학교 디지털콘텐츠공학과

- Serialize 처리 부분

```
...
using Newtonsoft.Json;

public class TestDataManager : MonoBehaviour {
    People p1 = new People("이용환", 50);

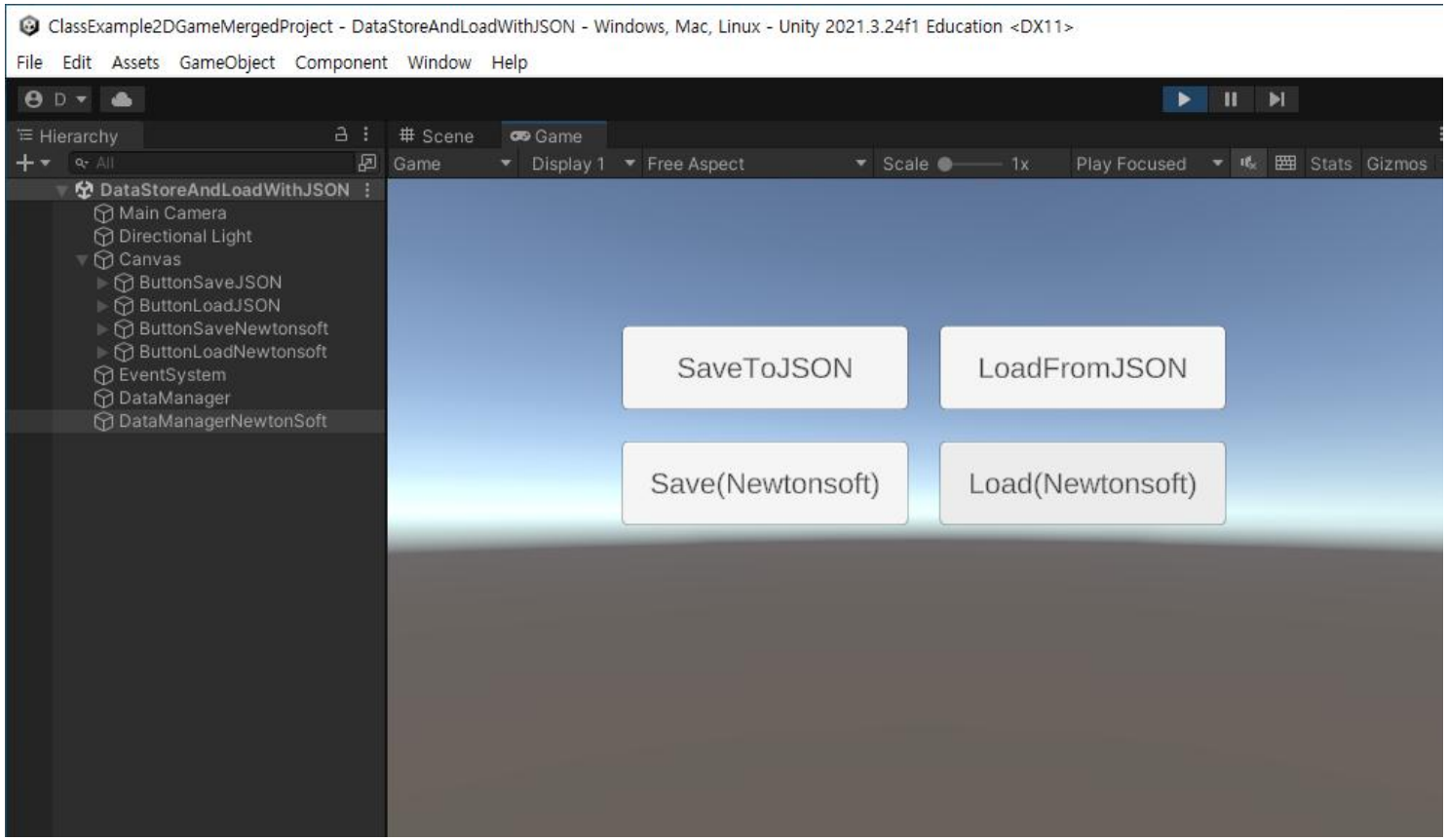
    // Start is called before the first frame update
    void Start() {
        string jsonFileName = Path.Combine(Application.dataPath, “Object2Json.json”);
        string readJsonData = File.ReadAllText(jsonFileName);
        People p2 = JsonConvert.DeserializeObject<People>(readJsonData);
        p2.PrintData();
    }

    // Update is called once per frame
    void Update() {}
}

public class People {
    public string name;
    public int age;

    public People(string name, int age) {
        this.name = name;
        this.age = age;
    }
    public void PrintData() {
        Debug.Log($”name={name}”);
        Debug.Log($”age={age}”);
    }
}
```

CopyRight ® 원광대학교 디지털콘텐츠공학과



CopyRight ® 원광대학교 디지털콘텐츠공학과

- Class 선언 부분

```
...
// newtonsoft JSON 관련 기능을 사용하기 위해
using Newtonsoft.Json;
using System.IO;

public class DataManagerWithNewtonSoftJson : MonoBehaviour {
    public JsonDataClass jsonData;

    // Start is called before the first frame update
    void Start() { jsonData = new JsonDataClass(); }

    // Update is called once per frame
    void Update() {}
}

[System.Serializable]
public class JsonDataClass {
    public int intNum;
    public float floatNum;
    public double doubleNum;
    public string str;
    public bool boolVar;
    public int[] intArray;
    public string[] strArray;
    public List<int> intList = new List<int>();
    public Dictionary<string, int> dicGameScore = new Dictionary<string, int>();
    public Dictionary<string, string> dic = new Dictionary<string, string>();
    public Vector3Int vecInt;

    ...
}
```

CopyRight ® 원광대학교 디지털콘텐츠공학과

- Class 처리 부분 (생성자, 출력)

```
[System.Serializable]
public class JsonDataClass {
    ...
    // 생성자
    public JsonDataClass() {
        intNum = 100;
        floatNum = 3.4f;
        doubleNum = 123.456;
        str = "JSON Test String";
        boolVar = true;
        intArray = new int[] { 1, 2, 3, 4, 5, 6, 7 };
        strArray = new string[] { "LG", "KT", "NC", "SSG", "두산" };
        for (int i=0; i<5; i++) { intList.Add(i*10);  }
        dicGameScore.Add("LG", 1);
        dicGameScore.Add("KT", 2);
        dicGameScore.Add("NC", 3);
        dic.Add("WK", "Wonkwang University");
        dic.Add("WKDC", "Wonkwang Digital Content");
        dic.Add("WKGC", "Wonkwang Game Content");
        vecInt = new Vector3Int(1, 2, 3);
    }

    public void PrintData()
    {
        Debug.Log($"integer = {intNum}");
        Debug.Log($"float = {floatNum}");
        Debug.Log($"double = {doubleNum}");
        Debug.Log($"str = {str}");
        Debug.Log($"boolVar = {boolVar}");
        for (int i=0; i<intArray.Length; i++) { Debug.Log($"int Array[{i}] = {intArray[i]}"); }
        for (int i = 0; i < strArray.Length; i++) { Debug.Log($"string Array[{i}] = {strArray[i]}"); }
        foreach(var item in intList) { Debug.Log($"int List: {item}"); }
        foreach(var item in dicGameScore) { Debug.Log($"int Dictionary: key={item.Key} /
value={item.Value}");}
        foreach (var item in dic) {Debug.Log($"string Dictionary: key={item.Key} / value={item.Value}");}
        Debug.Log($"int Vector = {vecInt}");
    }
```

CopyRigh

- Class 처리 부분

```
...
// newtonsoft JSON 관련 기능을 사용하기 위해
using Newtonsoft.Json;
using System.IO;

public class DataManagerWithNewtonSoftJson : MonoBehaviour {
    public JsonDataClass jsonData;

    // Start is called before the first frame update
    void Start() { jsonData = new JsonDataClass(); }

    // Update is called once per frame
    void Update() {}


    // 문자열로 만든 Json 데이터를 파일로 저장
    public void WriteToJsonFile() {
        string savingObjectToJson = JsonConvert.SerializeObject(jsonData);
        string savingJsonFileName = Path.Combine(Application.dataPath, "ObjectToJson.json");
        File.WriteAllText(savingJsonFileName, savingObjectToJson);
    }

    // 저장된 JSON 파일을 읽어서 오브젝트로 변환
    public void ReadFromJsonFile() {
        string jsonFileName = Path.Combine(Application.dataPath, "ObjectToJson.json");
        string readJsonData = File.ReadAllText(jsonFileName);
        JsonDataClass createObjectFromJson = JsonConvert.DeserializeObject<JsonDataClass>(readJsonData);
        createObjectFromJson.PrintData();
    }
}
```

CopyRight ® 원광대학교 디지털콘텐츠공학과

정리

- Unity JSON Utility
  - 네임스페이스 추가 필요 없음 (UnityEngine)
  - Object to Json : JsonUtility.ToJson(데이터);
  - Json to Object : JsonUtility.FormJson<클래스형식>(데이터);

- Newtonsoft.Json
  - Newtonsoft.Json 네임스페이스 추가
  - Object to Json : JsonConvert.SerializeObject(데이터);
  - Json to Object : JsonConvert.DeserializeObject<클래스형식>(데이터);

cf) Unity JSON Utility는 이중배열 변환을 지원하지 않지만, Newtonsoft.Json 에서는 지원

```
TempJson.json
{
   “scores” : [ [0,1], [2,3,4] ]
}
```

CopyRight ® 원광대학교 디지털콘텐츠공학과